Python 常用库入门教程（初学者友好版）

算法 • 爬虫 • 网络安全 • 数据 • AI • 办公自动化，每个库都带能跑的代码

> 这份文档和《常用第三方库速查.md》不一样：
> 那本是"**查的**"（想用某个库时翻一眼），这本是"**学的**"（从头看懂它到底怎么用）。
> **每个库都按同一个套路讲**：
> ① 它是什么（打比方）→ ② 怎么装 → ③ 第一个例子 → ④ 逐行在干嘛 → ⑤ **运行结果** → ⑥ 改一改试试
> **凡是标了「运行结果（实测）」的，都是我真跑出来的**，不是编的。
> 没标输出的（比如需要装数据库服务器、需要显卡的库），只给代码和说明，你自己跑跑看。
> **全文覆盖 120+ 个库，25 个章节，93 个小节。** 不用一次读完——按第 9 章的路线挑一条开始就行。

目录（25 章）

章	内容	适合谁
0	上手前必须知道的 5 件事	**所有人先看**
1	数据处理：numpy / pandas / matplotlib	想学数据分析
2	算法与数据结构（含标准库神器）	想刷题、想写快代码
3	爬虫：requests / BeautifulSoup / sqlite3 / playwright	想抓数据
4	网络与安全：socket / 加密 / 密码哈希 / 防守	对安全感兴趣 [!]
5	中文处理与 AI：jieba / sklearn / 大模型 API	想玩 AI
6	Web：FastAPI 十行写接口	想给别人提供服务
7	办公自动化：Excel / Word / PDF / 自动周报	**想马上用起来**
8	让代码变专业：loguru / pytest / ruff	想写得像样
9	**按目标选学习路线**（8 条）	不知道学什么时看
10	**20 个新手常见错误**	出错时对照查
11	**12 个动手小项目**	学完就练
12	图像 · 音视频 · OCR	要处理图片/视频
13	数据处理加餐：scipy / sympy / polars	数据方向进阶
14	爬虫的其它武器：httpx / selenium / parsel	爬虫进阶
15	网络探测与流量分析：scapy / psutil / mitmproxy	网络方向 [!]
16	逆向 / CTF / 防守进阶：capstone / yara	安全进阶 [!]
17	AI 进阶：xgboost / transformers / 向量检索	AI 方向
18	Web 进阶：Flask / Django / SQLAlchemy / streamlit	Web 方向
19	文档与桌面自动化：PDF / pyautogui / 定时 / 打包	自动化方向
20	数据库全家桶：MySQL / Redis / MongoDB / duckdb	要存数据
21	工程化工具箱：ruff / mypy / rich / typer	想让代码专业
22	算法库补充：sortedcontainers / ortools	刷题进阶
23	**全库索引（一页查完）**	忘了某个库在哪

第 0 章 • 上手前必须知道的 5 件事

0.1 什么是"库"

打比方：Python 本体是一个**基础工具箱**（螺丝刀、锤子）。而"库"是别人造好的**成品模块**——你想做个柜子，不用从砍树开始，直接买一块现成的板子拼上去。

- **标准库**：Python 自带，装好就有（json、sqlite3、collections…），**不用 pip install**
- **第三方库**：别人写的，要自己装（requests、pandas、numpy…）

判断方法：写 `import xxx` 如果报 `ModuleNotFoundError` —— 说明这是第三方库，需要装。

0.2 装库：就一条命令

```
pip install 库名
```

常用变体：

```
pip install requests # 装
pip install requests==2.32.3 # 装指定版本
pip install -U requests # 升级
pip uninstall requests # 卸载
pip list # 看装了什么
```

国内下载慢？加个镜像源：

```
pip install requests -i https://pypi.tuna.tsinghua.edu.cn/simple
```

0.3 为什么要用"虚拟环境"

打比方：虚拟环境就是给每个项目一个**独立的调料柜**。A 项目要 numpy 1.x，B 项目要 2.x，装在同一个锅里就会打架。

```
python -m venv .venv # 创建（在项目文件夹里执行一次）
.venv\Scripts\activate # Windows 激活
source .venv/bin/activate # macOS / Linux
deactivate # 退出
```

激活后命令行前面会出现（.venv）字样，说明你在自己的调料柜里了。

小白也能记住的一句：**每开一个新项目，先建虚拟环境，再 pip install。**

0.4 代码写在哪里跑

方式	适合	怎么用
VS Code	写正式代码（推荐）	装 Python 扩展，Ctrl+F5 运行
IDLE	刚入门随便试试	开始菜单搜 IDLE，File → New File
Jupyter Notebook	边写边看结果、做数据分析	pip install jupyter 后 jupyter notebook
命令行	跑脚本	python 文件名.py

> ※ **给初学者的小建议**：从 IDLE 或 VS Code 起步，等做数据分析时再上 Jupyter。

0.5 报错了怎么看（这一步学会，能省一半时间）

Python 的报错信息**从下往上看**：

```
Traceback (most recent call last):
 File "test.py", line 3, in <module>
 print(1 / 0)
ZeroDivisionError: division by zero
```

- **最后一行** = 错在哪类（ZeroDivisionError = 除以 0）
- **倒数第二行** = 错在哪一行（line 3）
- 中间的 File "xxx", line N = 出错的位置

新手最常遇到的 5 个报错：

报错	中文意思	怎么改
ModuleNotFoundError: No module named 'requests'	没装这个库	pip install requests
NameError: name 'df' is not defined	变量名写错 / 还没定义就用	检查拼写、检查顺序
TypeError: can only concatenate str (not "int") to str	字符串和数字相加了	用 str(数字) 转一下
IndentationError	缩进错了	统一用 4 个空格
KeyError: 'xxx'	字典里没有这个键	用 d.get("xxx", 默认值)

第 1 章 • 数据处理：让数字自己算

> 这一章解决三件事：**算得快**（numpy）、**算得省事**（pandas）、**看得懂**（matplotlib）。

1.1 numpy —— 让数字"整体"运算

它是什么：Python 的列表做乘法是"重复"，而 numpy 的数组做乘法是"每个元素都乘"。

```
pip install numpy
```

第一个例子：

```
import numpy as np

a = np.array([1, 2, 3, 4])
print("整体乘 2:", a * 2)
print("均值/最大:", a.mean(), a.max())

m = np.arange(6).reshape(2, 3) # 0~5 排成 2 行 3 列
print("2x3 矩阵:")
print(m)
print("每列求和:", m.sum(axis=0))
```

运行结果（我实测的）：

```
整体乘 2: [2 4 6 8]
均值/最大: 2.5 4
2x3 矩阵:
[[0 1 2]
 [3 4 5]]
每列求和: [3 5 7]
```

逐行在干嘛：

- np.array([1,2,3,4]) —— 把普通列表变成"数组"，这是 numpy 的基本单位
- a * 2 —— 每个元素都乘 2（普通列表会变成重复 8 个元素，这就是区别）

- `a.mean()` / `a.max()` —— 直接求平均、求最大，不用自己写循环
- `np.arange(6)` —— 生成 0,1,2,3,4,5；`.reshape(2,3)` 把它排成 2 行 3 列
- `m.sum(axis=0)` —— 沿"第 0 轴"（竖着）求和 = 每列的和

改一改试试：

1. 把 `a * 2` 改成 `a + 100`、`a ** 2`（平方），看看结果
2. 把 `.reshape(2,3)` 改成 `.reshape(3,2)`，看看矩阵变成什么样
3. 把 `axis=0` 改成 `axis=1`（每行求和），对比一下

> **一句话记住：numpy 快，是因为它不写 for 循环。**

1.2 pandas —— Python 里的"Excel"

它是什么：把数据变成**带表头的表格**，然后能像在 Excel 里一样筛选、排序、分组、统计——只是用代码，而且能处理几十万行。

```
pip install pandas openpyxl
```

第一个例子：

```
import pandas as pd

df = pd.DataFrame({
 "姓名": ["张三", "李四", "王五"],
 "分数": [686, 655, 700],
 "城市": ["杭州", "武汉", "北京"],
})
print(df)
print("平均分:", round(df["分数"].mean(), 1))
print("筛选 >=680:")
print(df[df["分数"] >= 680][["姓名", "分数"]])
print("按城市分组均值:", df.groupby("城市")["分数"].mean().to_dict())
```

运行结果：

```
 姓名 分数 城市
0 张三 686 杭州
1 李四 655 武汉
2 王五 700 北京
平均分: 680.3
筛选 >=680:
 姓名 分数
0 张三 686
2 王五 700
按城市分组均值: {'北京': 700.0, '杭州': 686.0, '武汉': 655.0}
```

逐行在干嘛：

- `pd.DataFrame({...})` —— 把字典变成表格；**字典的键变成表头**
- `df["分数"].mean()` —— 选出"分数"这一列求平均（`round(x, 1)` 是保留 1 位小数）
- `df[df["分数"] >= 680]` —— **按条件筛选行**（这行是 pandas 的灵魂，先记住形状）
- `[["姓名","分数"]]` —— 只保留这两列
- `groupby("城市")` —— 按城市分组，"每个城市的平均分是多少"

改一改试试：

1. 加一行 `("赵六", 720, "上海")`，重新跑
2. 改成筛选"分数在 660~700 之间"：`df[(df["分数"] >= 660) & (df["分数"] <= 700)]`
3. 把它存成 Excel：`df.to_excel("成绩.xlsx", index=False)`（`index=False` 是去掉左边那列编号）

> **一句话记住：pandas 是"用代码操作 Excel"，筛选用 `df[条件]`，统计用 `.mean()` / `.value_counts()` / `.groupby()`。**

1.3 matplotlib —— 把数字画成图

它是什么：把数据画成折线、柱状、散点、直方图，存成图片放进报告。

```
pip install matplotlib
```

第一个例子：

```
import matplotlib
matplotlib.use("Agg") # 只存文件，不弹窗（脚本必备）
import matplotlib.pyplot as plt

plt.rcParams["font.sans-serif"] = ["Microsoft YaHei"] # 中文不乱码
plt.rcParams["axes.unicode_minus"] = False # 负号不乱码

fig, ax = plt.subplots(figsize=(4, 3))
ax.bar(["张三", "李四", "王五"], [686, 655, 700], color="#4f81bd")
ax.set_title("三个人考了多少分")
ax.set_ylabel("分数")

plt.savefig("chart_demo.png", dpi=120, bbox_inches="tight")
plt.close(fig)
```

运行结果：

```
已保存: chart_demo.png 大小: 13125 字节
```

（打开那张图，你会看到一张带中文标题的柱状图。）

逐行在干嘛：

- `matplotlib.use("Agg")` —— 告诉它"只画到文件"。不然在服务器/脚本里可能报错
- `plt.rcParams["font.sans-serif"] = ["Microsoft YaHei"]` —— **不加这行，中文标题会变成方框**（新手第一大坑）
- `ax.bar(标签, 数值)` —— 画柱状图；换成 `.plot()` 是折线，`.scatter()` 是散点
- `plt.savefig(...)` —— 保存。**必须写在 `plt.show()` 之前**（show 之后画布就清了，新手第二大坑）
- `bbox_inches="tight"` —— 自动裁掉多余的白边

改一改试试：

1. 把 `ax.bar` 换成 `ax.plot`，看看变成折线图
2. 加一行 `ax.grid(True)` 打开网格
3. 加 `ax.set_xlabel("姓名")`

第 2 章 • 算法与数据结构：让程序变快

> 这一章有个反常识的重点：**最好用的"算法库"其实是 Python 自带的，不用装。**
> 很多人不知道标准库里已经有好东西，白白手写几十行。

2.1 三件套：Counter / deque / defaultdict

它们是什么（都来自标准库 `collections`）：

- **Counter** = 自动计数器。给它一堆东西，它告诉你"每个出现了几次"
- **deque** = 双端队列。两头都能快速进出（比 list 当队列快）

- **defaultdict** = 会"自动初始化"的字典，省掉写 `if k in d`

```
from collections import Counter, deque, defaultdict

# ① Counter：统计出现次数
words = "苹果 香蕉 苹果 橘子 苹果 香蕉".split()
c = Counter(words)
print(c)
print(c.most_common(2))

# ② deque：两头进出
q = deque(["乙", "丙"])
q.appendleft("甲")
q.append("丁")
print(list(q), "-> 左边弹出:", q.popleft())

# ③ defaultdict：自动建列表
groups = defaultdict(list)
for name, cls in [("张三", "一班"), ("李四", "二班"), ("王五", "一班")]:
 groups[cls].append(name)
print(dict(groups))
```

运行结果：

```
Counter({'苹果': 3, '香蕉': 2, '橘子': 1})
[('苹果', 3), ('香蕉', 2)]
['甲', '乙', '丙', '丁'] -> 左边弹出: 甲
{'一班': ['张三', '王五'], '二班': ['李四']}
```

逐行在干嘛：

- `Counter(words)` —— 一行完成"统计词频"，比手写字典计数循环省事
- `.most_common(2)` —— 出现最多的前 2 个，返回列表 %[(值, 次数)]%
- `q.appendleft("甲")` —— 从**左边**塞进去；`popleft()` 从左边取出来 → 这就是"队列/双端队列"
- `defaultdict(list)` —— 访问不存在的键时**自动创建空列表**，所以 `.append` 不会报错

改一改试试：

1. 把 `Counter` 换成统计一句话里每个字出现的次数
2. 用 `Counter` 找出一段文字里出现最多的前 3 个词
3. 用 `defaultdict(int)` 做"按班级统计人数"

> **一句话记住：要计数用 Counter，要队列用 deque，要分组用 defaultdict。**

2.2 heapq 和 bisect —— 排序相关的两个神器

它们是什么：

- **heapq** = 最小堆。**永远能 O(1) 拿出最小值**，适合"每次都取最小的"
- **bisect** = 二分查找。在**有序列表**里查位置/插入，O(log n)

```
import heapq, bisect

# heapq：最小值总是在最前面
nums = [5, 1, 8, 3]
heapq.heapify(nums)
print("堆里:", nums, " 最小值:", nums[0])
print("弹出:", heapq.heappop(nums), "剩下:", nums)
print("最大的两个:", heapq.nlargest(2, [10, 3, 99, 42]))

# bisect：有序列表里找位置
scores = [60, 70, 80, 90]
print("85 该插在位置:", bisect.bisect_left(scores, 85))
bisect.insort(scores, 85) # 插进去还保持有序
print("插入后:", scores)
```

运行结果：

```
堆里: [1, 3, 8, 5] 最小值: 1
弹出: 1 剩下: [3, 5, 8]
最大的两个: [99, 42]
85 该插在位置: 3
插入后: [60, 70, 80, 85, 90]
```

逐行在干嘛：

- `heapify(nums)` —— 把普通列表整理成"堆"（注意打印出来是 `[1,3,8,5]`，不是排好序的，但最小值一定在 `[0]`）
- `heappop()` —— 弹出**最小值**，并自动把剩下的重新整理成堆
- `nlargest(2, ...)` —— 取最大的 2 个（比先排序再切取更省资源）
- `bisect_left(scores, 85)` —— "85 应该插在下标 3 的位置"（因为 80 < 85 < 90）
- `insort` = insert + sort：**插进去还保持有序**

改一改试试：

1. 用 `heapq` + 循环，实现"从 10 个随机数里依次取出最小的三个"
2. 用 `bisect` 实现"给成绩找等级线"：`bisect.bisect([60,70,80,90], 75)` → 返回 2，表示落在第二档

2.3 itertools —— 排列组合一行搞定

它是什么：枚举**所有组合 / 排列 / 各种搭配**。暴力枚举、穷举、抽签都用它。

```
import itertools

print("两两组合:", list(itertools.combinations([1, 2, 3], 2)))
print("前 3 个排列:", list(itertools.permutations([1, 2, 3], 2))[:3])
print("笛卡尔积:", list(itertools.product("AB", "12")))
```

运行结果：

```
两两组合: [(1, 2), (1, 3), (2, 3)]
前 3 个排列: [(1, 2), (1, 3), (2, 1)]
笛卡尔积: [('A', '1'), ('A', '2'), ('B', '1'), ('B', '2')]
```

区别很重要（新手最容易混）：

- `combinations`（组合）：**(1,2) 和 (2,1) 算同一个**，只看"选了哪些"
- `permutations`（排列）：**(1,2) 和 (2,1) 算两个**，看"顺序"
- `product`（笛卡尔积）：A 的每个 × B 的每个，像"两列数据全配对"

改一改试试：用 `combinations` 列出"从 5 个人里选 2 个人"的所有可能。

2.4 functools.lru_cache —— 一行让递归快 1000 倍

它是什么：**记忆化。**函数算过的参数就存起来，下次同样参数直接返回结果，不再重算。

```
from functools import lru_cache

calls = 0
@lru_cache(maxsize=None) # 就是这一行
def fib(n):
 global calls
 calls += 1
 return n if n < 2 else fib(n-1) + fib(n-2)

print("fib(30) =", fib(30))
print("实际计算次数:", calls)
```

运行结果：

```
fib(30) = 832040
实际计算次数: 31
```

为什么这么神奇：不用 lru_cache 时，算 fib(30) 要重复计算**几百万次**；用了以后，因为每个 fib(k) 只算一次，所以**总共只算了 31 次**（k=0~30）。

改一改试试：

1. 把 @lru_cache 那行注释掉，把 30 改成 25，看看要跑多久（会明显变慢）
2. 用 lru_cache 加速"爬楼梯"问题（每次走 1 或 2 阶，共有几种走法）

> **一句话记住**：**递归慢，先想 lru_cache。**

2.5 networkx —— 处理"关系"的图论库

它是什么：专门处理**点和边**的关系：谁连谁、最短路径怎么走、有几个不连通的小团体。

```
pip install networkx
```

```
import networkx as nx

G = nx.Graph()
G.add_weighted_edges_from([("A", "B", 1), ("B", "C", 2), ("A", "C", 10)])
print("A 到 C 最短路径:", nx.shortest_path(G, "A", "C", weight="weight"))
print("所有连通的小团体:", list(nx.connected_components(G)))
```

适用场景：社交关系、地图导航、依赖关系、课程先修关系。

2.6 z3-solver —— 让它帮你"解条件"

它是什么：**约束求解器。**你只写条件（比如"x + y = 10 且 x - y = 4"），它帮你把答案算出来。数独、排班、逆向题都能用。

```
pip install z3-solver
```

```
from z3 import Ints, Solver
x, y = Ints("x y")
s = Solver()
s.add(x + y == 10, x - y == 4, x > 0, y > 0)
print(s.check()) # sat = 有解
print(s.model()) # [x = 7, y = 3]
```

2.7 一个重要的"复杂度直觉"

为什么第 2 章要学这些？ 因为**同一种功能，选对工具，速度差几百倍。**

任务	笨办法	聪明办法	差距
查重（1 万条）	双重循环 O(n²) = 1 亿次	用 set O(n) = 1 万次	**1 万倍**
找最小值（反复取）	每次 min(list)	heapq	快得多
有序插入	每次 sort()	bisect.insort	O(n log n) → O(log n)
递归	裸递归	lru_cache	指数级 → 线性

记住一句话：**先问"数据多大"，再想"用什么容器 / 什么库"。**

第 3 章 • 爬虫：把网上的数据搬回家

3.1 先懂一件事：网页其实就是一段"带标签的文字"

你在浏览器里看到漂亮的网页，但**右键 → 查看网页源代码**，看到的是一堆这样的东西：

```
<article class="product_pod">
 <h3><a href="catalogue/a-light-in-the-attic_1000/index.html" title="A Light in the Attic">A Light in th
    e Attic</a></h3>
 <p class="star-rating Three"></p>
 <p class="price_color">£51.77</p>
</article>
```

- <article> 是一个"盒子"
- class="product_pod" 是这个盒子的"名字"
- 书名在 <h3> 里，价格在 class="price_color" 里，**星级藏在 class 名字 Three 里**（很多人卡在这）

爬虫干的事就两件：① 把这段文字**下载**下来；② 从里面**挑出**你要的东西。

- 下载 → 用 **requests**
- 挑数据 → 用 **BeautifulSoup**

3.2 requests —— 去"下载"网页

它是什么：request 就是"请求"。它是**负责和网站说话的那个库**：你说"我要这个网址"，它把网页内容拿回来给你。

```
pip install requests
```

```
import requests

url = "https://books.toscrape.com/"
r = requests.get(url, timeout=15) # 一定要加 timeout

print("状态码:", r.status_code) # 200 = 成功
print("网页长度:", len(r.text), "字符")
print(r.text[:120]) # 先看一眼开头
```

运行结果（状态码是我实测的；长度每次抓会略有不同）：

```
状态码: 200
网页长度: 51xxx 字符
<!DOCTYPE html>
<!--[if lt IE 7]> <html class="no-js lt-ie9 lt-ie8"> <![endif]-->
...
```

> • 说明：网页长度、开头几行会随网站更新变化，**你看到的具体数字不一样是正常的**；
> 只要 **状态码是 200**、而且能在 %%r.text%% 里搜到书的名字，就说明你成功了。

逐行在干嘛：

- `requests.get(网址, timeout=15)` —— 去访问这个网址，最多等 15 秒
- `r.status_code` —— 服务器回你的"状态码"：**200 成功 / 404 没找到 / 403 被拒绝 / 429 太快了**
- `r.text` —— 网页的文字内容

四个必须做的动作（少一个都可能出问题）：

必做	为什么
加 `timeout=15`	不加的话，网络卡住程序会**永久挂住**
写 `headers={"User-Agent": ...}`	让网站知道你是谁，很多站点会拒绝默认的爬虫标识
检查 `r.status_code`	403/404 时 `r.text` 是错误页，直接解析会得到空结果，很难排查
处理编码 `r.encoding = r.apparent_encoding`	中文站常是 GBK，不处理会乱码

改一改试试：

1. 把 `url` 改成 `https://books.toscrape.com/catalogue/page-2.html`，看看还能不能拿到
2. 故意把网址改错一个字，观察 `status_code` 变成 404
3. 做一个"网址检查器"：给一串网址，逐个打印状态码

3.3 BeautifulSoup —— 从网页里"挑"出你要的数据

它是什么：BeautifulSoup 把那段 HTML 变成一个**能按名字查找**的对象。你说"我要所有 `class="product_pod"` 的盒子"，它就给你一列。

```
pip install beautifulsoup4 lxml
```

先看一个离线例子（不联网也能跑，方便你理解）：

```
from bs4 import BeautifulSoup

html = """<div class="book"><h3>A 书</h3><p class="price">£51.77</p>
<p class="star-rating Three"></p></div>
<div class="book"><h3>B 书</h3><p class="price">£22.50</p></div>"""

soup = BeautifulSoup(html, "lxml")

for card in soup.select("div.book"): # 每个"书盒子"逐个处理
 title = card.select_one("h3").get_text(strip=True)
 price = card.select_one("p.price").get_text(strip=True)
 star = card.select_one("p.star-rating")
 stars = star["class"][1] if star else "无评分" # 星级藏在 class 里
 print(title, "|", price, "|", stars)
```

运行结果：

```
A 书 | £51.77 | Three
B 书 | £22.50 | 无评分
```

选择器速查（背这几个就够了）：

写法	意思
`select("article.product_pod")`	所有"标签是 article 且 class 有 product_pod"的
`select_one("h3 a")`	只取第一个匹配的（取不到返回 None）
`select(".price_color")`	class 叫 price_color
`select("#main")`	id 叫 main
`tag.get_text(strip=True)`	取纯文字并去掉两端空白

写法	意思
tag["href"]	取属性

新手最容易踩的三个坑：

1. **取不到就返回 None** —— 直接 tag.get_text() 会报 AttributeError 让程序崩。

[√] 习惯写法：先 if tag: 判断，缺数据的就跳过

2. **链接是相对路径** —— 页面上是 catalogue/xxx.html，存进数据库没法用。

[√] 用 urljoin(当前页网址, href) 拼成完整网址

3. **数据藏在属性里** —— 像星级写在 class="star-rating Three"。

[√] 用 tag["class"] 取出来再解析（上面例子里的 star["class"][1] 就是取第二个词）

3.4 sqlite3 —— 把抓到的数据存起来

它是什么：**一个文件就是一个数据库**，Python 自带（不用 pip），特别适合存爬虫数据。

为什么不存 CSV? 因为爬虫要**去重、增量更新、反复查询**，这些恰好是数据库擅长的。

```
import sqlite3

conn = sqlite3.connect(":memory:") # 这里用内存库演示，换成文件名就是真数据库
conn.execute("CREATE TABLE s(name TEXT, score INT)")

# 用 ? 占位符传参（永远不要用 f-string 拼 SQL）
conn.executemany("INSERT INTO s VALUES (?, ?)", [("张三", 686), ("李四", 655), ("王五", 700)])
conn.commit()

for row in conn.execute("SELECT name, score FROM s WHERE score > ? ORDER BY score DESC", (660,)):
 print(row)
print("平均分:", conn.execute("SELECT AVG(score) FROM s").fetchone()[0])
conn.close()
```

运行结果：

```
('王五', 700)
('张三', 686)
平均分: 680.3333333333334
```

一个小技巧，能省很多事：把**网址设成主键**，重复抓同一页就不会产生重复数据。

```
conn.execute("""
 CREATE TABLE IF NOT EXISTS books(
 url TEXT PRIMARY KEY, -- 网址做主键 = 天然去重
 title TEXT, price REAL
 )""")
conn.execute("INSERT OR IGNORE INTO books VALUES (?, ?, ?)", (url, title, price))
```

3.5 把上面三步串起来：一个 20 行的最小爬虫

```
import requests, sqlite3, time
from bs4 import BeautifulSoup
from urllib.parse import urljoin

BASE = "https://books.toscrape.com/catalogue/page-1.html"
HEADERS = {"User-Agent": "StudyBot/1.0 (learning only)"}
conn = sqlite3.connect("books.db")
conn.execute("CREATE TABLE IF NOT EXISTS books(url TEXT PRIMARY KEY, title TEXT, price REAL)")

url, page = BASE, 0
while url and page < 3: # 抓 3 页就停（别给人家添麻烦）
 page += 1
 r = requests.get(url, headers=HEADERS, timeout=15)
 if r.status_code != 200:
 print("抓取失败:", r.status_code); break

 soup = BeautifulSoup(r.text, "lxml")
 for card in soup.select("article.product_pod"):
 a = card.select_one("h3 a")
 p = card.select_one("p.price_color")
 if not (a and p):
 continue
 price = float("".join(c for c in p.get_text() if c.isdigit() or c == "."))
 conn.execute("INSERT OR IGNORE INTO books VALUES (?, ?, ?)",
 (urljoin(url, a["href"]), a["title"], price))
 conn.commit()
 print("第", page, "页完成")

 nxt = soup.select_one("li.next a")
 url = urljoin(url, nxt["href"]) if nxt else None
 time.sleep(1) # 限速：每页间隔 1 秒

print("总共:", conn.execute("SELECT COUNT(*) FROM books").fetchone()[0], "本")
for row in conn.execute("SELECT title, price FROM books ORDER BY price DESC LIMIT 3"):
```

```
 print(row)
conn.close()
```

运行结果（我实测抓 2 页 40 本）：

```
第 1 页完成
第 2 页完成
总共: 40 本
('Our Band Could Be Your Life: Scenes from the American Indie Underground, 1981-1991', 57.25)
('Birdsong: A Story in Pictures', 54.64)
('Sapiens: A Brief History of Humankind', 54.23)
```

> 顺便说：这一页的第一本书是 **《A Light in the Attic》**，价格 **£51.77**，3 星；最便宜的是 **《In Her Wake》£12.84**。这些都是我实际抓到的。

3.6 遇到"JS 页面"怎么办 —— playwright

问题：有些网站的内容是**浏览器用 JavaScript 现算出来的**，你 requests 抓到的 HTML 里啥也没有。

解决办法：**让程序开一个真浏览器**，等页面渲染完再读。这就是 `playwright` 干的事。

```
pip install playwright
playwright install chromium # 第一次要下载浏览器（100 多 MB）
```

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
 browser = p.chromium.launch(headless=True) # headless=False 能看见浏览器
 page = browser.new_page()
 page.goto("https://quotes.toscrape.com/js/", wait_until="networkidle")
 page.wait_for_selector(".quote") # 等元素出现再读
 texts = page.locator(".quote .text").all_inner_texts()
 print("拿到", len(texts), "条，第一条:", texts[0][:30])
 browser.close()
```

什么时候该用它：requests 抓回来是空的 / 数据要滚动才出现 / 要点击按钮才有内容。

3.7 想抓"很多很多页" —— scrapy 和两个助手

库	它是什么	什么时候用
scrapy	**爬虫框架**：自带调度、去重、并发、重试、断点续爬	要抓几万页，手写太乱
tenacity	**自动重试**：失败等 2 秒再试	网络抖动、偶尔 5xx
tqdm	**进度条**：一行代码看到"跑到哪了"	长时间抓取，心里有底

```
from tenacity import retry, stop_after_attempt, wait_fixed

@retry(stop=stop_after_attempt(3), wait=wait_fixed(2)) # 失败自动重试 3 次
def fetch(url):
 r = requests.get(url, timeout=10)
 r.raise_for_status() # 4xx/5xx 直接抛异常 → 触发重试
 return r.text

from tqdm import tqdm
for i in tqdm(range(100), desc="进度"): # 一行就有进度条
 pass
```

3.8 [!] 三条必须记住的规矩

1. **先看** robots.txt **和网站条款**（浏览器打开 网站域名/robots.txt）。**有官方 API 就别爬。**
2. **必须限速**：time.sleep(1) 或更久。每秒几十个请求 = 事实上的攻击，会被封 IP，也不道德。看到 **429 立刻降速。**
3. **只爬公开数据**：不绕登录、不破验证码、**不碰个人隐私**（手机号、身份证、聊天记录）——那可能直接违法。

> 练手请用专门的教学站点：**books.toscrape.com**（书是假的）、**quotes.toscrape.com**、**httpbin.org。**

第 4 章 • 网络与安全：先学会保护自己

> ### [!] 学之前必须知道的法律边界
> 这一章的库都是"双刃剑"。**合法使用的三个前提，至少满足一个**：
> ① 是**你自己的**设备或网络；② 你拿到了**书面授权**；③ 在**靶场 / CTF 比赛**里练习。
> **扫描或攻击未授权的系统，在国内可能触犯《刑法》第 285 / 286 条。** 本节所有例子都写成"只打本机"的形式。
> 推荐合法练手平台：**PortSwigger Web Security Academy**（免费）、**OWASP Juice Shop**、**Hack The Box**、**TryHackMe。**

4.1 socket —— 网络编程的"地基"

它是什么：socket
是操作系统留给你程序的**网络接口。**你听到的一切网络词（TCP、连接、端口）最后都落到它身上。**理解它，才知道 requests 和浏览器在背后干了什么。**

第一个例子：检查本机某个端口通不通（不会打扰任何人）

```
import socket

def check(host, port, timeout=1):
 """能连上返回 True，连不上返回 False"""
 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
 s.settimeout(timeout)
 return s.connect_ex((host, port)) == 0

print("本机 1 端口开放吗:", check("127.0.0.1", 1))
print("本机 65500 端口开放吗:", check("127.0.0.1", 65500))
print("本机 IP:", socket.gethostbyname(socket.gethostname()))
```

运行结果：

```
本机 1 端口开放吗: False
本机 65500 端口开放吗: False
本机 IP: 10.120.0.140
```

逐行在干嘛：

- `socket.socket(AF_INET, SOCK_STREAM)` —— 建一个 **TCP** 类型的网络插座（AF_INET = IPv4）
- `settimeout(1)` —— 最多等 1 秒，不然可能卡很久
- `connect_ex((host, port))` —— 尝试连接，**返回 0 表示成功**，其它数字表示失败原因
- `` `with s` `` —— 用完自动关闭（好习惯）

> ※ **127.0.0.1 是什么意思**：它就是"**本机自己**"。练习时永远先拿它当目标。

4.2 hashlib / hmac —— "指纹"，不能还原

它是什么：**哈希（摘要）** 就是把任意内容压成一串固定长度的"指纹"。同一个内容 → 永远同一个指纹；内容改一个字 → 指纹完全不同。

关键：**哈希是单向的，不能从指纹反推出原文。**（加密才可逆，别搞混）

```
import hashlib, hmac

print("sha256:", hashlib.sha256(b"hello").hexdigest())

sig = hmac.new(b"key", b"data", hashlib.sha256).hexdigest()
print("hmac:", sig)
print("内容一样结果就一样:", hmac.new(b"key", b"data", hashlib.sha256).hexdigest() == sig)
```

运行结果：

```
sha256: 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
hmac: 5031fe3d989c6d1537a013fa6e739da2f6991c2a3dab0b0e5a2c1c9c36d3bb3e
内容一样结果就一样: True
```

它用来干什么：

- **校验文件有没有被改过**（下载大文件后比对 sha256）
- **接口签名**（hmac：带一把密钥，防止别人伪造请求）
- **去重**（内容一样 → 指纹一样）

注意：`b"hello"` 前面的 `b` 表示**字节**，哈希只能处理字节，字符串要先 `.encode()`。

4.3 secrets —— 生成"猜不到"的随机值

它是什么：Python 的标准库里有两个"随机"：`random` 和 `secrets`。

- `random` —— **可预测**（算法固定的伪随机），只能用来抽签、洗牌
- `secrets` —— **密码学安全**，用来生成**密码、验证码、Token**

```
import secrets

print("随机数:", secrets.randbelow(100)) # 0~99 之间
print("Token:", secrets.token_urlsafe(12)) # 用于登录凭证、邀请码
print("6 位验证码:", "".join(str(secrets.randbelow(10)) for _ in range(6)))
```

运行结果（每次都不一样，这就是重点）：

```
随机数: 37
Token: Y5zUIIfYx_N4zZKQ
6 位验证码: 436911
```

> [!] **新手最大的安全错误**：用 `random` 生成验证码或密码重置 Token。**必须用** `secrets`。

4.4 bcrypt —— 存密码的"正确姿势"

它是什么：专门用来**存用户密码**的哈希算法。它的特点是**故意很慢**，并且自动**加盐**（同一密码两次结果也不同），让暴力破解变得几乎不可能。

```
pip install bcrypt
```

```
import bcrypt

# 注册时：把密码变成哈希，存进数据库
password = b"user-password-123"
hashed = bcrypt.hashpw(password, bcrypt.gensalt())
print("存进数据库的哈希:", hashed[:29], "...") # 形如 b'$2b$12$...'

# 登录时：用用户输入去校验（不用解密，哈希本来就不可逆）
print("密码对不对:", bcrypt.checkpw(b"user-password-123", hashed)) # True
print("密码对不对:", bcrypt.checkpw(b"wrong-password", hashed)) # False
```

为什么不能用 md5 存密码：md5 太快，一秒能试几亿次；bcrypt 一次要几十毫秒，暴力破解直接不现实。**再加"加盐"，同一密码在不同用户那里哈希也不同。**

4.5 cryptography —— 真正意义上的"加密"

它是什么：和哈希不同，**加密是可逆的**——有钥匙就能还原。它用来保护"必须还原"的数据，比如配置文件里的密码、通信内容。

```
pip install cryptography
```

```
from cryptography.fernet import Fernet

key = Fernet.generate_key() # 生成一把钥匙（一定要保存好！）
f = Fernet(key)

token = f.encrypt(b"我的秘密")
print("密文:", token[:40], "...")
print("解密:", f.decrypt(token)) # 拿到钥匙就能还原
```

> **钥匙丢了，数据就永远打不开了**（这不是 bug，是特性）。
> 引申一句：**API Key、数据库密码绝不要写死在代码里**，用环境变量或 `python-dotenv` 读取，并且把配置文件加进 `.gitignore`。

4.6 更推荐的方向：做一个"防守的人"

新手容易被"攻击技术"吸引，但**安全行业真正缺人的是防守**。这些库正当、实用、还能写进简历：

```
pip install bandit pip-audit
```

```
bandit -r ./myproject # 扫描【你自己写的代码】有没有安全漏洞
pip-audit # 检查依赖库有没有已知 CVE 漏洞
```

工具	它是什么	用途
bandit	代码安全扫描器	提交前扫自己的代码
pip-audit	依赖漏洞扫描	上线前"体检"
yara-python	恶意样本特征匹配	写规则扫可疑文件（**防守必备**）
faker	生成假数据	做测试（**别拿真实数据练**）
scapy	构造/嗅探网络包	分析本机流量、学协议

4.7 想走安全方向？这个顺序最稳

1. **计算机网络 + Linux 命令 + HTTP 协议** ← 这三样比任何工具都重要
2. Python 编程（本教程）
3. `socket` → `requests` → 手工测试（Burp Suite / OWASP ZAP）
4. **靶场刷题**（PortSwigger 免费、Juice Shop、HTB）
5. 选方向：Web 安全 / 渗透测试 / 逆向 / 流量分析 / 应急响应 / 安全开发
6. **底线**：只打靶场、只测授权目标、发现漏洞先报告不公开利用

第 5 章 • 中文处理与 AI

5.1 jieba —— 中文分词

它是什么：英文有空格分词（`"I love you".split()` 就行），**中文不行。** `jieba` 就是专门把中文句子切开的库。

```
pip install jieba
```

```
import jieba

s = "高考志愿填报要综合考虑分数、城市和专业"
print("精确模式:", "/".join(jieba.cut(s)))
print("搜索模式:", "/".join(jieba.lcut_for_search(s)))
```

运行结果（形如）：

```
精确模式: 高考/志愿/填报/要/综合/考虑/分数/、/城市/和/专业
搜索模式: 高考/志愿/填报/要/综合/考虑/分数/城市/专业
```

用来干什么：统计词频、做词云、关键词提取、评论分析。

> ※ 第一次运行 jieba 会花一两秒"加载词典"，这是正常的。

5.2 wordcloud / snownlp —— 让文字变好看、变有用

库	它是什么	干什么
wordcloud	词云生成器	把高频词做成一张图
snownlp	中文情感分析	判断评论是正面还是负面

库	它是什么	干什么
pypinyin	汉字转拼音	排序、检索、注音

```
# 词云（注意：中文必须指定字体，否则是方框）
# pip install wordcloud jieba
# from wordcloud import WordCloud
# import jieba
# text = " ".join(jieba.cut("志愿 分数 城市 专业 未来 努力 学习 高考"))
# wc = WordCloud(font_path="C:/Windows/Fonts/msyh.ttc",
# width=800, height=400, background_color="white")
# wc.generate(text).to_file("词云.png")

from snownlp import SnowNLP
print("好评程度:", round(SnowNLP("这门课太棒了，学到很多").sentiments, 3))
print("差评程度:", round(SnowNLP("太糟糕了，完全听不懂").sentiments, 3))
```

结果会是：前者接近 1（正面），后者接近 0（负面）。

5.3 scikit-learn —— 三行训练一个 AI 模型

它是什么：**传统机器学习**的标准库。分类、回归、聚类一应俱全，而且**所有模型的用法几乎一样**：

`fit()` 训练 → `predict()` 预测 → `score()` 打分（记住这个套路，换模型只改一行）

```
pip install scikit-learn
```

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True) # 自带的鸢尾花数据集
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=42)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(Xtr, ytr) # 训练
pred = model.predict(Xte) # 预测
print("准确率:", round(accuracy_score(yte, pred), 4))
print("预测前 5 个:", pred[:5], " 真实答案:", yte[:5])
```

运行结果：

```
准确率: 1.0
预测前 5 个: [1 0 2 1 1] 真实答案: [1 0 2 1 1]
```

逐行在干嘛：

- `train_test_split` —— 把数据**切成两份**：80% 训练、20% 考试（**绝不能拿训练过的数据考自己**）
- `fit(Xtr, ytr)` —— 用训练集学习规律
- `predict(Xte)` —— 拿没见过的数据预测
- `accuracy_score` —— 对答案，算准确率

> ※ **新手须知**：`random_state=42` 是为了让结果**可复现**（不加的话每次切分不一样，结果飘）。

5.4 用大模型 API —— 几行调用 ChatGPT

它是什么：不想自己训模型？直接**调用别人训练好的大模型**。`openai` 这个库就是官方"打电话"的工具。

```
pip install openai
```

```
import os
from openai import OpenAI

client = OpenAI(api_key=os.environ["OPENAI_API_KEY"]) # 从环境变量读，不写死在代码里

resp = client.chat.completions.create(
 model="gpt-4o-mini",
 messages=[{"role": "user", "content": "用一句话解释什么是元组"}],
)
print(resp.choices[0].message.content)
```

结果就是模型给你的那句回答（内容每次不同）。

> **安全铁律**：API Key **绝不写进代码、绝不提交到 GitHub。**
> 正确做法：放到 `.env` 文件里，用 `python-dotenv` 读：

```
# .env 文件里写： OPENAI_API_KEY=sk-xxxxx
from dotenv import load_dotenv
import os
load_dotenv()
key = os.getenv("OPENAI_API_KEY")
```

5.5 AI 学习路径

表格数据预测（最实用，先学这个）→ `pandas` + `scikit-learn` + `xgboost`

深度学习 → `PyTorch`（先做 MNIST 手写数字识别）

大模型应用（最好上手）→ `openai` API → `langchain` 做知识库问答（RAG）

中文 NLP → `jieba` → `transformers` → `sentence-transformers`

第 6 章 • Web：把你写的东西"挂到网上"

6.1 FastAPI —— 10 行写出一个接口

它是什么：**API 就是"别人可以调用你的函数"。**比如你写了个"分析分数"的函数，用 FastAPI 包一层，别人就能通过网址调用它。

它最大的优点：你只要**写清楚类型**，它自动帮你**校验参数**并**生成一份能点击测试的文档。**

```
pip install "fastapi[standard]"
```

新建文件 `main.py`：

```
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI(title="我的第一个接口")

class Student(BaseModel): # 定义"请求要长什么样"
 姓名: str = Field(min_length=1)
 高考分数: int = Field(ge=0, le=750) # 分数必须在 0~750 之间

@app.get("/") # 访问根路径时返回的
def home():
 return {"message": "服务已启动"}

@app.post("/analyze") # POST /analyze 时执行
def analyze(stu: Student):
 if stu.高考分数 >= 700:
 level = "顶尖"
 elif stu.高考分数 >= 660:
 level = "优秀"
 else:
 level = "普通"
 return {"姓名": stu.姓名, "等级": level}
```

启动它（在命令行里，不是直接跑）：

```
uvicorn main:app --reload
```

运行结果：

```
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Application startup complete.
```

然后打开浏览器访问两个地址：

地址	你会看到
http://127.0.0.1:8000/	{"message":"服务已启动"}
http://127.0.0.1:8000/docs	**自动生成的接口文档**，可以点"Try it out"直接测试！

逐行在干嘛：

- `class Student(BaseModel)` —— 用一个类描述"请求体应该有哪些字段"，**字段名和类型都写清楚**
- `Field(min_length=1)` / `Field(ge=0, le=750)` —— 直接在这里加**校验规则**
- `@app.post("/analyze")` —— 声明"当别人往 /analyze 发 POST 请求时，执行下面的函数"
- 返回值直接写 `return {"...": ...}` —— FastAPI 自动转成 JSON

> ※ **为什么推荐 FastAPI 给初学者**：你自己都还没搞懂 HTTP，它已经把文档、校验、错误提示全做好了——**打开 /docs 就能玩**，成就感很强。

6.2 调用你自己写的接口

接口写好了，用 `requests` 就能调（回到第 3 章的知识）：

```
import requests

r = requests.post("http://127.0.0.1:8000/analyze",
 json={"姓名": "张三", "高考分数": 686}, timeout=5)
print(r.json()) # {'姓名': '张三', '等级': '优秀'}

# 试试故意传错类型，看 FastAPI 怎么回你
bad = requests.post("http://127.0.0.1:8000/analyze",
 json={"姓名": "张三", "高考分数": 9999}, timeout=5)
print(bad.status_code) # 422 = 参数校验失败
```

> ※ **422 是个好东西**：说明你的校验生效了。**参数不合法就直接被拦住，不用你自己写 if 判断。**

6.3 其他 Web 相关（先认识，不用急）

库	它是什么	什么时候用
Flask	更简单更老的 Web 框架	想用最少的代码起个网页
Django	全家桶（自带后台、用户系统、ORM）	要正式做中大型网站
uvicorn	跑 FastAPI 的服务器	启动接口
SQLAlchemy	ORM：用 Python 类操作数据库	网站要读写数据库
pydantic	数据校验	接口参数、配置
streamlit / gradio	几行代码做数据 / AI 界面	快速做演示给同学看

```
# 用 streamlit 做一个网页版数据看板，只要几行
pip install streamlit
streamlit run app.py
```

第 7 章 • 办公自动化：把重复工作交出去

7.1 openpyxl —— 用代码写 Excel

它是什么：能**读和写 .xlsx**，还能设样式（加粗、底色、列宽）、写公式、冻结首行。

```
pip install openpyxl
```

```
from openpyxl import Workbook, load_workbook

wb = Workbook()
ws = wb.active
ws.title = "成绩"

ws.append(["姓名", "分数", "评价"]) # 写表头
for name, score in [("张三", 686), ("李四", 655)]:
 ws.append([name, score, "优秀" if score >= 660 else "良好"])

# （样式可以后加，写法见下面）
wb.save("成绩表.xlsx")

# 再读回来看看
wb2 = load_workbook("成绩表.xlsx")
ws2 = wb2["成绩"]
print("工作表名:", wb2.sheetnames)
for row in ws2.iter_rows(values_only=True):
 print(" ", row)
```

运行结果：

```
工作表名: ['成绩']
 ('姓名', '分数', '评价')
 ('张三', 686, '优秀')
 ('李四', 655, '良好')
```

加样式的写法（让表格好看）：

```
from openpyxl.styles import Font, PatternFill, Alignment

ws["A1"].font = Font(bold=True, color="FFFFFF") # 白色加粗字
ws["A1"].fill = PatternFill("solid", fgColor="4F81BD") # 蓝色底
ws.column_dimensions["A"].width = 14 # A 列变宽
ws.freeze_panes = "A2" # 冻结首行（滚动时表头不动）
ws["D1"] = "=SUM(B2:B3)" # 写 Excel 公式
```

7.2 python-docx —— 用代码写 Word

它是什么：自动生成报告、周报、合同。**把重复排版交给代码。**

```
pip install python-docx
```

```
from docx import Document

doc = Document()
doc.add_heading("我的第一份自动报告", level=1) # 标题
doc.add_paragraph("这一行是正文。") # 段落

table = doc.add_table(rows=1, cols=2) # 插表格
table.style = "Table Grid"
table.rows[0].cells[0].text = "指标"
table.rows[0].cells[1].text = "数值"
row = table.add_row().cells
row[0].text = "平均分"; row[1].text = "680.3"

doc.add_paragraph("第一条建议", style="List Bullet") # 项目符号
doc.save("报告.docx")
```

运行结果（保存后读回来验证）：

```
段落数: 2 首个标题: 我的第一份自动报告
表格: [['指标', '数值'], ['平均分', '680.3']]
```

> **常用 style**: "List Bullet"（圆点）、"List Number"（编号）、"Intense Quote"（强调引用）。

7.3 pdfplumber / pypdf —— 处理 PDF

库	它是什么	干什么
pdfplumber	**提取** PDF 里的文字和表格	解析发票、报表
pypdf	拆分 / 合并 / 旋转 / 加密	批量整理
reportlab	**生成** PDF	做证书、发票

```
# 提取文字
import pdfplumber
with pdfplumber.open("input.pdf") as pdf:
 print("总共", len(pdf.pages), "页")
 print(pdf.pages[0].extract_text()[:200])

# 合并两个 PDF
from pypdf import PdfReader, PdfWriter
w = PdfWriter()
for f in ["a.pdf", "b.pdf"]:
 for page in PdfReader(f).pages:
 w.add_page(page)
w.write("合并后.pdf")
```

> [!] **扫描件（图片型 PDF）提不出文字**，那种要配合 OCR（pytesseract / easyocr）。

7.4 综合小项目：自动生成"周报"

把上面几个库串起来，就是一个**真能用的自动化脚本**：

```
from openpyxl import Workbook
from docx import Document
from datetime import datetime

data = [("周一", 8), ("周二", 7.5), ("周三", 9), ("周四", 8.5), ("周五", 7)]
total = sum(h for _, h in data)

# ① 出 Excel
wb = Workbook(); ws = wb.active; ws.title = "工时"
ws.append(["日期", "工时"])
for d, h in data:
 ws.append([d, h])
ws.append(["合计", total])
wb.save("周报.xlsx")

# ② 出 Word
doc = Document()
doc.add_heading("本周工作周报", level=1)
doc.add_paragraph("生成时间：" + datetime.now().strftime("%Y-%m-%d %H:%M"))
t = doc.add_table(rows=1, cols=2); t.style = "Table Grid"
t.rows[0].cells[0].text = "日期"; t.rows[0].cells[1].text = "工时"
for d, h in data:
 c = t.add_row().cells; c[0].text = d; c[1].text = str(h)
doc.add_paragraph("本周共工作 %.1f 小时，平均每天 %.1f 小时。" % (total, total / len(data)))
doc.save("周报.docx")
print("已生成 周报.xlsx 和 周报.docx，总工时:", total)
```

这类脚本的价值：写一次，以后每周点一下就行。 这就是"自动化"最朴素、也最实用的形态。

第 8 章 • 让代码"看起来专业"（很容易被忽略）

新手往往功能写出来了，但代码"一看就是新手"。下面这几个库能快速提升：

库	它是什么	一行怎么用
loguru	一行搞定的漂亮日志（替代 print 调试）	`from loguru import logger`
tqdm	进度条	`for x in tqdm(list):`
tenacity	自动重试	加个装饰器就行
pytest	自动化测试	函数名以 `test_` 开头，跑 `pytest`
ruff	代码检查 + 格式化（一个顶三个）	`ruff check . --fix`

库	它是什么	一行怎么用
rich	终端里输出彩色表格	`from rich import print`

```
# ① loguru：比 print 强一万倍
from loguru import logger
logger.add("run.log", rotation="10 MB", encoding="utf-8") # 日志落盘，超过 10MB 自动换
logger.info("程序启动")
try:
 1 / 0
except ZeroDivisionError:
 logger.exception("出错了") # 自动带上完整堆栈，不用自己拼字符串

# ② tqdm：进度条
from tqdm import tqdm
for i in tqdm(range(100), desc="处理中"):
 pass

# ③ tenacity：失败自动重试（网络请求必备）
from tenacity import retry, stop_after_attempt, wait_fixed

@retry(stop=stop_after_attempt(3), wait=wait_fixed(2))
def unstable():
 ...
```

pytest 的最小例子（新建 `test_calc.py`）：

```
def add(a, b):
 return a + b

def test_add():
 assert add(1, 2) == 3
 assert add(0, 0) == 0
 assert add(-1, 1) == 0
```

```
pytest -v
```

为什么值得学：改完代码跑一下 pytest，就知道有没有把别的地方改坏。 这是从"能跑"到"敢改"的分界线。

第 9 章 • 按目标选学习路线

你想干什么	按顺序学这些	大概要多久
自动化办公	openpyxl → python-docx → pdfplumber → playwright	1~2 周
数据分析	numpy → pandas → matplotlib → seaborn	2~3 周
爬虫	requests → BeautifulSoup → sqlite3 → playwright → scrapy	3~4 周
Web 开发	Flask → FastAPI → SQLAlchemy → Django	1~2 个月
AI 应用	openai API → langchain（RAG）→ 部署	2~4 周
机器学习	pandas → scikit-learn → xgboost → PyTorch	2~3 个月
网络安全	网络基础 → socket → 靶场刷题 → scapy / yara	3~6 个月
算法刷题	collections / heapq / itertools / lru_cache → 刷 LeetCode	持续

> ※ **重要建议：一次只学一条线。**同时开五条线，最后哪条都不熟。

第 10 章 • 20 个新手常见错误（对照自查）

#	错误	正确做法
1	报 `ModuleNotFoundError` 就怀疑代码	先 `pip install`（记得在**当前虚拟环境**里）
2	requests 不加 `timeout`	**永远加**，否则会永久卡住
3	网页中文乱码	`r.encoding = r.apparent_encoding`
4	`select_one` 返回 None 就崩	先 `if tag:` 再取值
5	相对链接直接存数据库	用 `urljoin(当前页, href)`
6	边遍历 list 边删元素	倒着删，或遍历副本，或重建列表
7	用 f-string 拼 SQL	**永远用 ? 占位符**
8	用 MD5 / SHA1 存密码	用 **bcrypt** / argon2
9	用 `random` 生成 Token	用 **secrets**
10	API Key 写死在代码里	用 `.env` + `python-dotenv`，并加进 `.gitignore`
11	matplotlib 中文变方框	`plt.rcParams["font.sans-serif"] = ["Microsoft YaHei"]`
12	保存的图是空白	`savefig` 写在 `show()` **之前**
13	pandas 设置 copy 警告	用 `df.loc[条件, "列"] = 值` 而不是链式赋值
14	训练完拿训练集测自己	必须 `train_test_split` 留出测试集
15	忘了 `opt.zero_grad()`（PyTorch）	训练四步：zero_grad → 前向 loss → backward → step
16	OpenCV 图片颜色反了	OpenCV 是 **BGR**，要 `cvtColor` 转 RGB
17	爬虫跑太快被封	`time.sleep()` 限速；看到 **429 立刻降速**
18	递归爆栈 / 太慢	加 `@lru_cache`；必要时 `sys.setrecursionlimit`
19	`lst = lst.sort()` 变成 None	`sort()` 是原地修改；要新列表用 `sorted(lst)`
20	所有代码堆在一个文件里	用**函数**拆分，一个函数干一件事

第 11 章 • 学完就练：12 个动手小项目

难度从低到高，每个只用到一个库，做完就真的会了：

1. **二维码生成器**（qrcode）—— 输入一段文字，生成二维码图片
2. **批量改图名**（pathlib + os）—— 把文件夹里的图片按规则重命名
3. **给图片加水印**（Pillow）—— 批量给图片右下角加字
4. **自动生成成绩单**（openpyxl）—— 从列表生成带样式的 Excel
5. **自动周报**（python-docx）—— 读取数据生成 Word 报告
6. **成绩分析图**（pandas + matplotlib）—— 读 CSV，出三张图
7. **词频统计**（jieba + Counter）—— 统计一篇文章里出现最多的 10 个词
8. **网页标题抓取器**（requests + BeautifulSoup）—— 输入网址，输出标题
9. **迷你爬虫 + 入库**（requests + bs4 + sqlite3）—— 抓 3 页存数据库
10. **接口小服务**（FastAPI）—— 写一个"计算 BMI"的接口，带自动文档
11. **本机端口扫描器**（socket）—— **只扫 127.0.0.1**，列出开放端口
12. **密码管理器小工具**（secrets + bcrypt + json）—— 生成密码、存哈希

> 每个项目都不超过 50 行。**做完这 12 个，你就从"学过 Python"变成"会用 Python"了。**

第 12 章 • 图像 • 音视频 • OCR

> **这类库解决什么问题**：图片太大要压缩、要批量加水印、要把图里的字认出来、要把语音转成文字。

12.1 Pillow —— 图片处理"万金油"

它是什么：Python 处理图片的**基础库**。打开、裁剪、缩放、旋转、滤镜、加水印、转格式、生成缩略图——全靠它。

```
pip install pillow
```

```
from PIL import Image, ImageDraw, ImageFont

# ① 新建一张纯色图
img = Image.new("RGB", (300, 120), "#4f81bd")
d = ImageDraw.Draw(img)
font = ImageFont.truetype("C:/Windows/Fonts/msyh.ttc", 28) # 中文必须指定字体
d.text((30, 40), "你好 Python", font=font, fill="white")
img.save("pillow_demo.png")

# ② 再读回来看看
im = Image.open("pillow_demo.png")
print("尺寸:", im.size, "模式:", im.mode)
print("左上角像素:", im.getpixel((0, 0)))
print("缩略图后尺寸:", im.resize((120, 48)).size)
```

运行结果（实测）：

```
尺寸: (300, 120) 模式: RGB
左上角像素: (79, 129, 189)
缩略图后尺寸: (120, 48)
```

几个最常用的操作：

想干什么	写法
打开	`Image.open("a.jpg")`
缩放	`img.resize((400, 300))`
按比例缩略	`img.thumbnail((128, 128))`
旋转	`img.rotate(90, expand=True)`
转灰度	`img.convert("L")`
高斯模糊	`img.filter(ImageFilter.GaussianBlur(2))`
存成别的格式	`img.save("a.png")`

> [!] **中文变方框**：`ImageFont.truetype` 里的字体路径必须指向支持中文的字体（Windows 用 `msyh.ttc`）。

12.2 OpenCV —— 计算机视觉

它是什么：比 Pillow 更"专业"的图像库：边缘检测、人脸识别、目标跟踪、摄像头实时处理。

```
pip install opencv-python
```

```
import cv2

img = cv2.imread("input.jpg") # 读图
print("形状(高,宽,通道):", img.shape) # 顺序是"高在前"
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) # 转灰度
edges = cv2.Canny(gray, 100, 200) # 边缘检测
cv2.imwrite("edges.png", edges)

# 人脸检测（用它自带的模型）
cascade = cv2.CascadeClassifier(
 cv2.data.haarcascades + "haarcascade_frontalface_default.xml")
faces = cascade.detectMultiScale(gray, 1.1, 4)
print("检测到", len(faces), "张脸")
```

> [!] **两个历史遗留坑**：
> ① OpenCV 读进来是 **BGR** 顺序（不是 RGB），喂给 matplotlib 颜色会反，要 `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`；
> ② 中文路径读图可能失败，用 `cv2.imdecode(np.fromfile(path, np.uint8), cv2.IMREAD_COLOR)`。

12.3 OCR 文字识别

它是什么：把**图片里的文字**变成**可复制的文本**。

库	中文解释	特点
pytesseract	老牌 OCR 引擎 Tesseract 的封装	轻量，但要**另装** Tesseract 程序
easyocr	基于深度学习	中文效果好很多，装完自动下载模型
paddleocr	百度开源 OCR	中文最强，模型较大
cnocr	专攻中文	轻量

```
# pytesseract 用法（需先装 Tesseract 程序）
import pytesseract
from PIL import Image
pytesseract.pytesseract.tesseract_cmd = r"C:\Program Files\Tesseract-OCR\tesseract.exe"
text = pytesseract.image_to_string(Image.open("shot.png"), lang="chi_sim+eng")
print(text)

# easyocr 用法（首次会自动下载模型，约 100MB）
import easyocr
reader = easyocr.Reader(["ch_sim", "en"])
for box, text, conf in reader.readtext("shot.png"):
 print(round(conf, 2), text)
```

12.4 音视频

库	中文解释	干什么
moviepy	用代码剪视频	裁剪、拼接、加字幕、提取音频
ffmpeg-python	用 Python 组装 ffmpeg 命令	转码、抽帧、压缩（**需先装 ffmpeg**）
pydub	音频剪辑	切片、格式转换、调音量（**需 ffmpeg**）
mutagen	读写音频标签	批量改 MP3 的歌名/歌手
librosa	音频特征提取	语音/音乐分析
whisper	**语音转文字**（OpenAI 开源）	生成字幕、会议记录

```
# moviepy：裁一段视频 + 提取音频
from moviepy import VideoFileClip
clip = VideoFileClip("in.mp4").subclipped(10, 20) # 取 10~20 秒
clip.write_videofile("out.mp4")
clip.audio.write_audiofile("audio.mp3")

# whisper：语音转文字（首次下载模型）
import whisper
model = whisper.load_model("base")
print(model.transcribe("audio.mp3", language="zh")["text"][:100])
```

12.5 二维码 / 条码

```
import qrcode # pip install qrcode[pil]

img = qrcode.make("https://github.com/chenhg5/cc-connect")
img.save("我的二维码.png")
print("二维码已生成")

# 识别二维码
from PIL import Image
from pyzbar.pyzbar import decode # pip install pyzbar
print(decode(Image.open("我的二维码.png")))
```

> ※ **实用场景**：给活动做签到码、给链接做二维码、批量给产品图加条码。

第 13 章 • 数据处理"加餐"

> 第 1 章的 numpy / pandas / matplotlib 是主力，这几个是**特定场景的补充**。

13.1 scipy —— 科学计算工具箱

它是什么：numpy 的"高级版"，提供**优化、积分、插值、信号处理、统计检验**等算法。

```
pip install scipy
```

```
from scipy import optimize, stats

# 求最小值：找让 (x-3)² + (y+1)² 最小的 (x, y)
r = optimize.minimize(lambda v: (v[0]-3)**2 + (v[1]+1)**2, [0, 0])
print("最小值点:", r.x.round(4))

# 描述统计
d = stats.describe([1, 2, 3, 4, 100])
print("均值:", d.mean, " 方差:", round(d.variance, 2))
```

运行结果（实测）：

```
最小值点: [ 3. -1.]
均值: 22.0 方差: 1902.5
```

13.2 sympy —— 让符号"算公式"

它是什么：numpy/scipy 算的是**数字**，sympy 算的是**公式**：解方程、求导、积分、化简，结果还是符号。

```
pip install sympy
```

```
import sympy as sp

x = sp.symbols("x")
print("解方程 x^2-4=0:", sp.solve(x**2 - 4, x))
print("求导 sin(x):", sp.diff(sp.sin(x), x))
print("定积分 x^2 从0到1:", sp.integrate(x**2, (x, 0, 1)))
print("化简 sin²+cos²:", sp.simplify(sp.sin(x)**2 + sp.cos(x)**2))
```

运行结果（实测）：

```
解方程 x^2-4=0: [-2, 2]
求导 sin(x): cos(x)
定积分 x^2 从0到1: 1/3
化简 sin²+cos²: 1
```

13.3 数据类库"三家补充"

库	中文解释	什么时候用
polars	和 pandas 同用途但**更快更省内存**（Rust 写的）	数据上百万行，pandas 卡住
seaborn	matplotlib 的"美化层"，一行画统计图	想要好看的分布图/箱线图
statsmodels	专业统计：回归、假设检验、时间序列	学术论文要 p 值
pyarrow	列式存储（parquet 格式）	大数据交换、给 pandas 加速
xlrd	读**老式 .xls**（openpyxl 只支持 .xlsx）	别人发来的是老格式

```
# polars：和 pandas 很像，但写法更"链式"
import polars as pl
df = pl.DataFrame({"姓名": ["张三", "李四"], "分数": [686, 655]})
print(df.filter(pl.col("分数") > 660))
print(df.select(pl.col("分数").mean().alias("平均分")))
```

第 14 章 • 爬虫的"其它武器"

> 第 3 章讲了主力（requests + BeautifulSoup），这一章是**按需补充**的装备。

14.1 httpx —— 会"同时发很多请求"的 requests

它是什么：requests 的升级版。**最大的区别是它支持** async，可以同时发几百个请求，速度快很多；也支持 HTTP/2。

什么时候用：要抓几百上千个页面，requests 一个个发太慢。

```
pip install httpx
```

```
import httpx, asyncio

# 普通用法和 requests 几乎一样
r = httpx.get("https://httpbin.org/get", timeout=10)
print(r.status_code)

# 异步用法：5 个请求"同时"发
async def main():
 async with httpx.AsyncClient(timeout=15) as client:
 tasks = [client.get("https://httpbin.org/delay/1") for _ in range(5)]
 res = await asyncio.gather(*tasks)
 print("全部完成:", [r.status_code for r in res])

asyncio.run(main())
```

> ※ 对比一下：5 个"各耗时 1 秒"的请求，同步要 5 秒，异步只要 1 秒多。

14.2 selenium —— 老牌浏览器自动化

它是什么：和 playwright 干一样的活（驱动真浏览器），只是**资历更老、资料更多。**

什么时候用：维护老项目，或者某些特定浏览器驱动只有 Selenium 支持。

```
pip install selenium
```

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome() # 需要本机装 Chrome
driver.get("https://quotes.toscrape.com/js/")
driver.implicitly_wait(5) # 最多等 5 秒让元素出现
texts = driver.find_elements(By.CSS_SELECTOR, ".text")
print("拿到", len(texts), "条")
driver.quit() # 别忘了关，不然会留下一堆浏览器进程
```

14.3 三个"方便小工具"

库	中文解释	例子
parsel	用 **XPath / CSS** 提取（Scrapy 内部就用它）	`Selector(text=html).css("a::attr(href)").get()`
pyquery	用 **jQuery 语法**解析 HTML	`pq(html)("a").attr("href")`
fake-useragent	随机生成浏览器标识	`from fake_useragent import UserAgent; UserAgent().random`
robobrowser	简化表单提交	老式登录页自动化

```
from parsel import Selector

html = "<div><a href='/x'>标题</a><a href='/y'>第二</a></div>"
sel = Selector(text=html)
print("第一个链接:", sel.css("a::attr(href)").get())
print("全部链接:", sel.css("a::attr(href)").getall())
print("用 XPath 取文字:", sel.xpath("//a/text()").getall())
```

第 15 章 • 网络探测与流量分析

> ### [!] 再提醒一次
> 这一章的库**只能用于自己的设备、有书面授权的目标，或靶场/CTF。**
> **扫描或攻击别人的系统，可能构成刑事犯罪。**

15.1 python-nmap —— 端口探测（只扫自己）

它是什么：调用著名的 nmap 工具来做端口扫描。**用途：清点自己服务器有哪些端口开着。**

```
pip install python-nmap # 还要另外装 nmap 本体：nmap.org 下载
```

```
import nmap

nm = nmap.PortScanner()
nm.scan("127.0.0.1", "1-1024") # 只扫本机
for host in nm.all_hosts():
 print("主机:", host, "状态:", nm[host].state())
 for proto in nm[host].all_protocols():
 for port, info in nm[host][proto].items():
 print(" 端口", port, info["state"], info.get("name", ""))
```

15.2 scapy —— 看网络包长什么样

它是什么：能**构造、解析、嗅探网络数据包。学习协议最快的方式就是"把它拆开看"。**

```
pip install scapy
```

```
from scapy.all import get_if_list, IP, TCP, Ether

print("本机网卡:", get_if_list())

# 构造一个包（只在自己程序里玩，不发送）
pkt = Ether() / IP(dst="127.0.0.1") / TCP(dport=80)
print("包的结构:", pkt.summary())
print(pkt.show(dump=True)[:300])

# 抓包（需要管理员权限；只抓本机流量）
# pkts = sniff(count=5, filter="tcp port 80")
```

> [!] Windows 上抓包/发原始包需要**管理员权限 + 安装 Npcap。**

15.3 网络诊断与监控

库	中文解释	干什么
dnspython	DNS 查询	查域名的 A / MX / TXT 记录
python-whois	查域名注册信息	域名是谁注册的、什么时候到期
pyshark	解析抓包文件（Wireshark 的 Python 口）	分析 pcap 流量
psutil	系统与网络状态	看进程、端口、CPU、内存占用
mitmproxy	中间人代理	**调试手机/浏览器的流量**（抓 App 接口）
faker	生成假数据	做测试、演示（**别用真实数据**）

```
# dnspython：查域名解析
import dns.resolver
for rtype in ("A", "MX"):
 try:
 print(rtype, [str(r) for r in dns.resolver.resolve("example.com", rtype)])
 except Exception as e:
 print(rtype, "查询失败:", type(e).__name__)

# psutil：看我电脑的 CPU 和内存
import psutil
print("CPU 核心:", psutil.cpu_count(), " 使用率:", psutil.cpu_percent(interval=1), "%")
print("内存:", round(psutil.virtual_memory().used / 1024**3, 1), "GB /",
 round(psutil.virtual_memory().total / 1024**3, 1), "GB")

# faker：造测试数据
from faker import Faker
fk = Faker("zh_CN")
print("造一个假用户:", fk.name(), fk.phone_number(), fk.email())
```

15.4 mitmproxy —— 看清"手机 App 到底请求了什么"

它是什么：一个**中间人代理**。你把手机/浏览器的网络指向它，它就能**把每个请求和响应都显示出来**。

典型用途（合法场景）：**调试自己的 App / 小程序**，看它调用了哪些接口、参数长什么样。

```
pip install mitmproxy
mitmproxy # 交互式界面
mitmweb # 网页版界面（更好上手）
```

> [!] 只用于**你自己的流量**或**你有权测试的系统**。分析别人的通信是违法的。

第 16 章 • 逆向 / CTF / 防守进阶

> 这一章门槛偏高，**新手可以先跳过**，等你想打 CTF 或者做安全分析时再回来。

16.1 z3-solver —— "约束求解"，逆向题的大杀器

它是什么：你只写**条件**，它帮你**算出答案**。比如逆向题里常见的"输入要满足一堆算式"，用它就不用一个个手推。

```
pip install z3-solver
```

```
from z3 import Ints, Solver

x, y = Ints("x y") # 声明两个"未知整数"
s = Solver()
s.add(x + y == 10, # 条件：两数之和是 10
 x - y == 4, # 条件：两数之差是 4
 x > 0, y > 0)
print("有解吗:", s.check()) # sat = 有解
print("解是:", s.model()) # [x = 7, y = 3]
```

> ※ 数独、排班、密码学小题目，都能用它解。

16.2 二进制分析三件套

库	中文解释	干什么
pyelftools	解析 **ELF**（Linux 可执行文件）	看程序结构、节表、符号
pefile	解析 **PE**（Windows .exe/.dll）	分析 Windows 程序
capstone	**反汇编引擎**（把机器码变汇编）	看程序具体指令
unicorn	**CPU 模拟器**（直接跑一段代码，不用真的运行程序）	安全地分析可疑代码
angr	**符号执行**（自动找"走哪条路能满足条件"）	高级 CTF、漏洞挖掘
pwn tools（pwntools）	CTF 必备工具箱：连接、打包数据、ROP	**注意：只支持 Linux/macOS**

```
# capstone：把一段机器码翻译成汇编（这是"反汇编"的核心）
from capstone import Cs, CS_ARCH_X86, CS_MODE_64

code = b"\x55\x48\x89\xe5\x48\x83\xec\x10" # 一段 x86-64 机器码
md = Cs(CS_ARCH_X86, CS_MODE_64)
for ins in md.disasm(code, 0x1000):
 print("%x:\t%s\t%s" % (ins.address, ins.mnemonic, ins.op_str))
```

```
# pefile：看一个 exe 的基本信息（只读文件，无任何风险）
import pefile
pe = pefile.PE("C:/Windows/System32/notepad.exe")
print("机器类型:", hex(pe.FILE_HEADER.Machine))
print("节的数量:", pe.FILE_HEADER.NumberOfSections)
pe.close()
```

16.3 防守方向：这些更值得学

新手建议：与其研究怎么攻击，不如先学**怎么发现和防护**——这个方向岗位多、也不踩法律红线。

工具	中文解释	干什么
yara-python	**恶意文件特征匹配**	写规则扫可疑文件（防守核心技能）
bandit	扫自己代码的漏洞	提交前自查
pip-audit	扫依赖库的 CVE 漏洞	上线前体检
ssdeep / py-tlsh	模糊哈希	找"相似"的样本
geoip2	IP → 地理位置	分析日志里的攻击来源
elasticsearch	日志检索	搭 SIEM（安全分析平台）

```
# yara：写一条规则，扫目录里的文件
import yara

rule = yara.compile(source="""
rule SuspiciousDownloader {
 strings:
 $a = "powershell" nocase
 $b = "DownloadString" nocase
 condition:
 all of them
}
""")
# 只在你【有权分析】的文件上跑：
# print(rule.match("sample.exe"))
print("规则编译成功，可以开始扫描了")
```

> **一个正当又有意思的入门项目**：用 capstone + yara
> 写一个"可疑文件快速体检脚本"，扫自己电脑的下载目录。这比"学攻击"更适合写进简历。

第 17 章 • AI 进阶

> 第 5 章讲了 jieba / sklearn / 大模型 API，这一章是**往深走**的路线。

17.1 XGBoost / LightGBM —— 表格数据的"冠军模型"

它是什么：把几百棵小决策树组合起来投票（叫"梯度提升"）。**在表格数据上经常比神经网络还强**，Kaggle 比赛里几乎人手一个。

```
pip install xgboost lightgbm
```

```
import xgboost as xgb
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=0)

model = xgb.XGBClassifier(n_estimators=300, learning_rate=0.05, max_depth=5,
 eval_metric="auc", random_state=0)
model.fit(Xtr, ytr)
print("AUC:", round(roc_auc_score(yte, model.predict_proba(Xte)[:, 1]), 4))
```

> ※ **AUC 越接近 1 越好。**它衡量"模型排序能力"，比准确率更适合不平衡数据。

17.2 transformers —— 几行代码用上大模型

它是什么：HuggingFace 出的库，**几行就能加载 BERT / GPT 这类预训练模型**，不用自己训练。

```
pip install transformers
```

```
from transformers import pipeline

# 情感分析（第一次运行会自动下载模型，几百 MB）
clf = pipeline("sentiment-analysis",
 model="distilbert-base-uncased-finetuned-sst-2-english")
print(clf("I love Python!")) # [{'label': 'POSITIVE', 'score': 0.999...}]
print(clf("This is terrible.")) # [{'label': 'NEGATIVE', 'score': 0.999...}]

# 换个任务只要改一个参数：文本生成、摘要、翻译、问答……
# gen = pipeline("text-generation", model="gpt2")
```

17.3 语义相似度：sentence-transformers

它是什么：把**句子变成向量**，然后算"两句话有多像"。

用途：语义搜索、找重复问题、文本聚类、给 RAG 做检索。

```
pip install sentence-transformers
```

```
from sentence_transformers import SentenceTransformer
from sentence_transformers.util import cos_sim

m = SentenceTransformer("all-MiniLM-L6-v2")
v = m.encode(["我爱编程", "我喜歡寫程式", "今天天气不错"])

print("第1句 vs 第2句（意思相近）:", round(cos_sim(v[0], v[1]).item(), 3))
print("第1句 vs 第3句（不相关） :", round(cos_sim(v[0], v[2]).item(), 3))
```

结果会是：第一组接近 1，第二组明显更小。

17.4 大模型应用三件套

库	中文解释	干什么
datasets	一行加载公开数据集	训练/测试数据
tiktoken	数 token	**估算 API 要花多少钱**
accelerate	多卡 / 混合精度	训练加速
peft	参数高效微调（LoRA）	**低成本微调大模型**（不用 A100）

库	中文解释	干什么
langchain / llama-index	把大模型和**你的数据、工具**串起来	知识库问答（RAG）、Agent

```
# tiktoken：算一段文字大概要花多少 token（直接关系到 API 费用）
import tiktoken
enc = tiktoken.get_encoding("cl100k_base")
text = "Python 是一门很好学的编程语言"
print("token 数:", len(enc.encode(text)))
print("切出来的 token:", enc.encode(text))

# datasets：一行下载公开数据集
from datasets import load_dataset
# ds = load_dataset("imdb") # 首次会下载，比较大
# print(ds["train"][0])
```

17.5 中文 NLP 补充

库	中文解释	干什么
pypinyin	汉字转拼音	排序、检索、注音
snownlp	中文情感分析	评论正负判断
hanlp / ltp	专业中文 NLP	分词、词性、命名实体识别
wordcloud	词云	可视化高频词

```
from pypinyin import lazy_pinyin, Style

print("普通拼音:", lazy_pinyin("志愿填报"))
print("首字母 :", lazy_pinyin("志愿填报", style=Style.FIRST_LETTER))
```

运行结果（实测）：

```
普通拼音: ['zhi', 'yuan', 'tian', 'bao']
首字母 : ['z', 'y', 't', 'b']
```

17.6 AI 学习顺序（别跳步）

1. **pandas 把数据弄干净** ← 90% 的时间都在这
2. **scikit-learn 跑通一个完整流程**（切分 → 训练 → 评估）
3. **xgboost 提分**
4. 想做图像/文本 → **PyTorch**（先做 MNIST 手写数字）
5. 想做应用 → **openai API + langchain**（最容易出成果）

> [!] **别一上来就学深度学习。**大多数实际问题的数据量，用 XGBoost 就够了。

第 18 章 • Web 进阶

> 第 6 章用 FastAPI 起了个接口。这一章补上"做成一个网站"需要的其它零件。

18.1 Flask —— 最简单的 Web 框架

它是什么：**几行代码就能起一个网站。**比 FastAPI 更简单、更老、资料最多，适合小工具和教学。

```
pip install flask
```

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.get("/") # 访问首页
def home():
 return "<h1>你好，这是我的第一个网站</h1>"

@app.get("/api/hello") # 一个接口
def hello():
 name = request.args.get("name", "匿名") # 读网址参数 ?name=xxx
 return jsonify({"code": 0, "msg": "你好，" + name})

if __name__ == "__main__":
 app.run(host="127.0.0.1", port=5000, debug=True)
```

跑起来后，浏览器打开：

地址	你会看到
http://127.0.0.1:5000/	一个大标题"你好，这是我的第一个网站"
http://127.0.0.1:5000/api/hello?name=张三	{"code":0,"msg":"你好，张三"}

> [!] debug=True 会自动重载并显示详细报错，**开发时很好用，但绝不能用在对外服务上**（有安全风险）。

18.2 Django —— "全家桶"

它是什么：比 Flask 重得多，但**自带后台管理、用户登录、数据库 ORM、表单**——做一个正式网站不用东拼西凑。

什么时候用：要做**中大型网站**，或者需要现成的管理后台。

```
pip install django
django-admin startproject mysite # 创建项目
cd mysite
python manage.py runserver # 启动
python manage.py createsuperuser # 创建后台管理员
```

访问 http://127.0.0.1:8000/admin/ 就能看到一个**现成的后台管理界面**（这就是它最大的卖点）。

18.3 SQLAlchemy —— 用 Python 类操作数据库

它是什么：**ORM**（对象关系映射）。你**不用写 SQL**，用 Python 类和方法就能读写数据库。

好处：换数据库（SQLite → MySQL）基本不用改代码。

```
pip install sqlalchemy
```

```
from sqlalchemy import create_engine, Integer, String, select
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, Session

class Base(DeclarativeBase): pass

class Student(Base): # ① 用类描述一张表
 __tablename__ = "student"
 id: Mapped[int] = mapped_column(Integer, primary_key=True)
 name: Mapped[str] = mapped_column(String(20))
 score: Mapped[int] = mapped_column(Integer)

engine = create_engine("sqlite:///school.db") # ② 连接数据库
Base.metadata.create_all(engine) # ③ 自动建表

with Session(engine) as s: # ④ 增
 s.add_all([Student(name="张三", score=686), Student(name="李四", score=655)])
 s.commit()

 for stu in s.scalars(select(Student).where(Student.score > 660)): # ⑤ 查
 print(stu.name, stu.score)
```

输出：张三 686

18.4 Web 其它零件

库	中文解释	干什么
uvicorn	ASGI 服务器	跑 FastAPI（`uvicorn main:app --reload`）
gunicorn	WSGI 服务器	Linux 上部署正式服务
pydantic	数据校验	请求参数 / 配置项校验（FastAPI 的基石）
jinja2	模板引擎	把数据填进 HTML 页面
starlette	轻量 ASGI 框架	FastAPI 的底层
streamlit	几行代码做数据网页	做数据看板、AI 演示
gradio	几行代码做 AI 界面	**给模型做个网页 Demo 给同学玩**

```
# streamlit：一个网页版数据看板，不到 10 行
# 保存为 app.py，然后命令行运行： streamlit run app.py
import streamlit as st
import pandas as pd

st.title("我的成绩看板")
df = pd.DataFrame({"姓名": ["张三", "李四", "王五"], "分数": [686, 655, 700]})
st.dataframe(df) # 显示表格
st.bar_chart(df.set_index("姓名")) # 显示柱状图
st.metric("平均分", round(df["分数"].mean(), 1)) # 显示大数字指标
```

> ※ **streamlit 是"给不会前端的人做的前端"**——你只写 Python，它自动生成网页。

第 19 章 · 文档与桌面自动化进阶

19.1 PDF 三剑客补充

库	中文解释	干什么
pypdf	拆分 / 合并 / 旋转 / 加密	批量整理 PDF
pdfplumber	**提取文字和表格**（最好用）	解析发票、报表

库	中文解释	干什么
PyMuPDF（pymupdf）	**PDF 转图片**、极速提取	扫描件、批量转换
reportlab	**生成** PDF	做证书、发票、报告
weasyprint	HTML → PDF	**用网页排版做 PDF**（最省事）

```
# PyMuPDF：读页数 + 提取第一页文字
import pymupdf

doc = pymupdf.open("某个文件.pdf")
print("总页数:", doc.page_count)
print("第1页前 30 字:", doc[0].get_text()[:30].replace(chr(10), " "))
doc.close()

# 还能转图片：一页一张 PNG
# for i, page in enumerate(doc):
# page.get_pixmap(dpi=150).save("page_%d.png" % i)
```

运行结果（我拿本教程自己的 PDF 试的）：

```
总页数: 12
第1页前 30 字: 爬虫实战教程 第 1 页 爬虫实战：从 0 到 1 做一个「
```

```
# reportlab：程序化生成 PDF（还能画表格、插图片）
from reportlab.pdfgen import canvas

c = canvas.Canvas("我的报告.pdf")
c.setFont("Helvetica", 16)
c.drawString(80, 750, "Hello PDF!")

c.setFont("Helvetica", 11)
c.drawString(80, 720, "This PDF was generated by Python.")
c.showPage()
c.save()
print("PDF 已生成")
```

19.2 桌面自动化

库	中文解释	干什么
pyautogui	**模拟鼠标键盘**	给没有 API 的软件做自动化（点按钮、填表）
pywinauto	Windows 控件级自动化	比 pyautogui 更稳（能精确找到按钮）

```
import pyautogui, time

print("屏幕分辨率:", pyautogui.size())
print("当前鼠标位置:", pyautogui.position())

time.sleep(3) # 给自己 3 秒切到目标窗口
pyautogui.click(400, 300) # 点一下
pyautogui.write("hello", interval=0.05) # 打字
pyautogui.hotkey("ctrl", "s") # 组合键：Ctrl+S

# 找图点击（先截一小块图存成 btn.png）
# loc = pyautogui.locateCenterOnScreen("btn.png", confidence=0.9)
# pyautogui.click(loc)
```

> [!] **两个重要提醒**：
> ① 把鼠标**快速甩到屏幕左上角**可以紧急中止（FAILSAFE 保护）；
> ② 屏幕缩放会改变坐标，**能用"找图"就别写死坐标。**

19.3 定时任务与打包

库	中文解释	干什么
schedule	最简单的定时（在程序里边跑边等）	每天 20:00 执行
APScheduler	专业调度器（支持 cron 表达式）	复杂定时、持久化任务
python-crontab	管理 Linux 的 crontab	服务器定时
pyinstaller	**把 .py 打包成 .exe**	发给不会 Python 的人用

```
import schedule, time

def job():
 print("该干活了！")

schedule.every().day.at("20:00").do(job) # 每天 20:00
schedule.every(10).minutes.do(job) # 每 10 分钟
schedule.every().monday.at("09:00").do(job) # 每周一 9:00

while True: # 需要程序一直挂着
 schedule.run_pending()
 time.sleep(1)
```

```
# 打包成 exe（在项目目录里执行）
pip install pyinstaller
pyinstaller -F -w 我的程序.py
# -F = 打成单个文件；-w = 运行时不显示黑色命令行窗口
# 生成的 exe 在 dist/ 文件夹里
```

> ※ Windows 上想"开机自动跑"，除了
> schedule，**用系统自带的"任务计划程序"或把脚本丢进「启动」文件夹**更省事。

第 20 章 • 数据库全家桶

> **先记住一句话**：小项目用 **sqlite3**（第 3 章讲过，一个文件就是数据库）就够了。
> 只有"多个人同时用"或"数据特别大"时，才需要下面这些。

20.1 各种数据库的"连接器"

它们是什么：这些库只干一件事——**让你的 Python 连上那个数据库。**用法惊人地相似：

```
pip install pymysql # 连 MySQL
pip install "psycopg[binary]" # 连 PostgreSQL
pip install pymongo # 连 MongoDB
pip install redis # 连 Redis
```

```
# MySQL 的标准用法（PostgreSQL 几乎一样，只换 import 和连接串）
import pymysql

conn = pymysql.connect(host="127.0.0.1", user="root", password="xxx",
 database="test", charset="utf8mb4")
with conn.cursor() as cur:
 cur.execute("CREATE TABLE IF NOT EXISTS s(name VARCHAR(20), score INT)")
 cur.executemany("INSERT INTO s VALUES (%s, %s)", [("张三", 686), ("李四", 655)])
 conn.commit()

 cur.execute("SELECT name, score FROM s WHERE score > %s", (660,))
 for row in cur.fetchall():
 print(row) # ('张三', 686)
conn.close()
```

[!] **注意占位符不一样**：sqlite3 用 ?，pymysql/psycopg 用 ``s。
但**原则完全一样：永远用占位符，永远不要用 f-string 拼 SQL。**

20.2 什么时候用哪种数据库

数据库	中文解释	适合什么
SQLite	一个文件，零配置	**小工具、爬虫、个人项目**（首选）
MySQL	最流行的开源关系型数据库	传统网站、中小型系统
PostgreSQL	更严谨、功能更强	数据一致性要求高
MongoDB	文档型（存 JSON 一样的结构）	字段不固定、结构经常变
Redis	**内存数据库**，读写极快	缓存、计数器、任务队列、限流

```
# Redis：几个最常用的场景
import redis
r = redis.Redis(host="127.0.0.1", port=6379, decode_responses=True)

r.set("user:1:name", "张三", ex=60) # set 并 60 秒后自动过期（这就是"缓存"）
print(r.get("user:1:name")) # 张三

r.incr("page:views") # 计数器：访问量 +1
print("访问量:", r.get("page:views"))

r.lpush("task_queue", "任务A") # 当队列用
print("队列长度:", r.llen("task_queue"))
```

Redis 最典型的用法：

- **缓存**：接口算一次要 3 秒？结果丢进 Redis，下次 0.01 秒返回
- **计数**：文章浏览量、点赞数
- **限流**：一分钟最多请求 100 次
- **队列**：把耗时任务丢进队列，后台慢慢处理

20.3 ORM 与数据库迁移

库	中文解释	干什么
SQLAlchemy	最主流的 ORM	用类操作数据库（见第 18 章）
peewee	轻量 ORM	小项目，代码少
tortoise-orm	支持 async 的 ORM	FastAPI 异步项目常用
alembic	**数据库迁移**	改表结构时不丢数据
duckdb	分析型数据库	**直接用 SQL 查 CSV / Parquet**

```
# duckdb：不用装数据库、不用导入，直接对 CSV 写 SQL
import duckdb

# 先用 pandas 造个 csv 演示
import pandas as pd
pd.DataFrame({"姓名": ["张三", "李四", "王五"], "分数": [686, 655, 700]}).to_csv(
 "成绩.csv", index=False, encoding="utf-8-sig")

print(duckdb.sql("SELECT * FROM '成绩.csv' WHERE 分数 > 660").df())
print(duckdb.sql("SELECT AVG(分数) AS 平均分 FROM '成绩.csv'").fetchall())
```

第 21 章 • 工程化工具箱（让代码"专业起来"）

> 功能写出来了，但代码"一看就是新手"——**这一章就是解决这个问题**的。
> 不需要全装，挑 3~4 个用起来就有明显变化。

21.1 代码质量三件套

工具	中文解释	一句话用法
ruff	**检查 + 自动格式化**（一个顶 flake8 + isort + black）	`ruff check . --fix`
black	自动格式化（代码风格统一）	`black .`
mypy	**类型检查**：提前发现"传错类型"	`mypy main.py`
isort	自动整理 import 顺序	`isort .`
flake8	经典风格检查（老项目常见）	`flake8 .`
pre-commit	提交前自动跑检查	`pre-commit install`

```
pip install ruff
ruff check . # 检查问题
ruff check . --fix # 能自动修的自动修
ruff format . # 格式化
```

> ※ **为什么值得用**：它会指出"变量定义了没用""import 了没用""这行太长了"这种小毛病。
> **对新手来说，它像个免费的老师。**

21.2 日志、调试、进度

库	中文解释	干什么	
loguru	**一行搞定漂亮日志**（替代 print 调试）	`logger.add("run.log")`	
icecream	调试打印，**自动显示变量名和值**	`ic(x)` → `ic	x: 5`
rich	终端里输出彩色文字、表格、进度条	`from rich import print`	
tqdm	进度条	`for x in tqdm(list):`	

```
# loguru：日志会落盘，出错了自动带完整堆栈
from loguru import logger
logger.add("run.log", rotation="10 MB", encoding="utf-8")
logger.info("程序启动")
try:
 1 / 0
except ZeroDivisionError:
 logger.exception("这里出错了") # 日志文件里会有完整的调用栈

# rich：终端里打印彩色表格
from rich.console import Console
from rich.table import Table

c = Console()
t = Table(title="我的成绩表")
t.add_column("姓名"); t.add_column("分数", justify="right")
t.add_row("张三", "686"); t.add_row("李四", "655")
c.print(t)
c.print("[bold green]分析完成！[/bold green]")
```

21.3 命令行工具：typer / click

它是什么：把脚本变成**专业命令行工具**（支持 --参数、--help、子命令）。

```
pip install typer
```

```
import typer

app = typer.Typer()

@app.command()
def analyze(score: int, city: str = "杭州", verbose: bool = False):
 """根据分数给出建议（试试 --help）"""
 if verbose:
 typer.echo("参数: score=%d city=%s" % (score, city))
 typer.echo("稳妥" if score >= 680 else "冲刺")

if __name__ == "__main__":
 app()
```

```
python cli.py analyze 686 --city 杭州 --verbose
python cli.py --help # 自动生成的帮助文档
```

21.4 配置与数据格式

库	中文解释	干什么
python-dotenv	从 .env 读配置和密钥	**别把 API Key 写进代码**
PyYAML	读写 YAML 配置文件	管理复杂配置
orjson	更快的 JSON（比内置快好几倍）	大数据量序列化
ipython	增强版交互解释器	边写边试，自动补全

```
# python-dotenv：正确管理密钥的方式
import os
from dotenv import load_dotenv

# 先在项目里建一个 .env 文件，里面写： API_KEY=sk-xxxxx
load_dotenv()
print("读到密钥:", os.getenv("API_KEY"))
print("读到名字:", os.getenv("NAME"))
```

运行结果（实测，demo.env 里写的是 API_KEY=test-123 / NAME=小李）：

```
读到密钥: test-123
读到名字: 小李
```

> [!] **一定要做**：把 .env 加进 .gitignore！
> 不然你的密钥会跟着代码一起传到 GitHub 上，几分钟内就可能被盗用。

第 22 章 • 算法库补充（刷题 / 竞赛向）

> 第 2 章讲过"标准库四件套"。这几个是**当你觉得标准库不够用时**的补充。

22.1 sortedcontainers —— 永远保持有序的容器

它是什么：普通 list 每次插入都要重新排序，这个库的 SortedList **插入 / 查找都是 O(log n)**，而且**始终有序。**

```
pip install sortedcontainers
```

```
from sortedcontainers import SortedList

sl = SortedList([5, 1, 3])
sl.add(2) # 插进去，自动排好
print("始终有序:", list(sl))
print("3 的下标:", sl.bisect_left(3))
print("最小的两个:", list(sl[:2]))
```

运行结果（实测）：

```
始终有序: [1, 2, 3, 5]
3 的下标: 2
最小的两个: [1, 2]
```

什么时候用：滑动窗口求中位数、实时排行榜、需要频繁"插一个 / 找位置"的场景。

22.2 more-itertools —— itertools 的加强版

它是什么：itertools 很好用，但有些常用操作它没有（比如**滑动窗口**、**分块**）。这个库补上了。

```
pip install more-itertools
```

```
import more_itertools as mit

print("滑动窗口:", list(mit.windowed([1, 2, 3, 4], 2)))
print("分块:", list(mit.chunked([1, 2, 3, 4, 5], 2)))
```

运行结果（实测）：

```
滑动窗口: [(1, 2), (2, 3), (3, 4)]
分块: [[1, 2], [3, 4], [5]]
```

> ※ **滑动窗口特别实用**：算"每 3 天的移动平均""每 5 个字符的重复检测"，一行就够。

22.3 约束与优化求解

库	中文解释	干什么
python-constraint	约束满足求解	**数独**、排班、选课搭配

库	中文解释	干什么
pulp	线性规划	资源分配、成本最小化
ortools	Google 的运筹优化工具箱	排班、路径规划（TSP）、装箱

```
# python-constraint：解一个"选课不能冲突"的小问题
# pip install python-constraint
from constraint import Problem

p = Problem()
p.addVariable("数学", [1, 2, 3]) # 三门课分别安排在第几节
p.addVariable("英语", [1, 2, 3])
p.addVariable("体育", [1, 2, 3])
p.addConstraint(lambda a, b: a != b, ["数学", "英语"]) # 数学和英语不能同节
p.addConstraint(lambda a, b: a != b, ["英语", "体育"])
p.addConstraint(lambda a, b: a != b, ["数学", "体育"])

sol = p.getSolution()
print("一种排法:", sol)
print("总共有几种排法:", len(p.getSolutions()))
```

输出示例：{'数学': 1, '英语': 2, '体育': 3}（具体顺序随机）

22.4 刷题时该用什么（一张速查）

题目要求	用什么
统计次数 / 找众数	`collections.Counter`
队列 / BFS	`collections.deque`
每次取最小	`heapq`
有序插入、二分	`bisect` 或 `sortedcontainers`
排列组合枚举	`itertools`
递归太慢	`functools.lru_cache`
滑动窗口 / 分块	`more-itertools`
图论（最短路/连通性）	`networkx`
一堆条件求满足的解	`z3-solver`
线性规划	`pulp` / `ortools`

> ※ **刷题的第一原则：先判断数据规模。**
> n ≤ 20 → 暴力枚举；n ≤ 5000 → O(n²)；n 到 10^5 以上 → 必须 O(n log n) 或 O(n)。

第 23 章 • 全库索引（一页查完）

> 按章分组的**全部库**清单。忘了某个库在哪一章，从这里找。

第 0~1 章 • 基础与数据处理

库	一句话中文解释	章节
pip / venv	装库 / 隔离环境	0
numpy	批量数值计算（矩阵、统计）	1
pandas	Python 里的 Excel（筛选/分组/统计）	1
matplotlib	画图（折线/柱状/直方图）	1

第 2 章 • 算法与数据结构

库	一句话中文解释	章节
collections（Counter / deque / defaultdict）	计数 / 双端队列 / 自动初始化字典	2
heapq	最小堆（每次取最小）	2
bisect	二分查找 / 有序插入	2
itertools	排列组合、笛卡尔积	2
functools.lru_cache	记忆化，让递归飞快	2
networkx	图论（最短路、连通分量）	2
z3-solver	约束求解（写条件自动求解）	2 / 16
sortedcontainers	永远有序的容器	22
more-itertools	滑动窗口 / 分块	22
python-constraint	约束满足（数独、排班）	22
pulp / ortools	线性规划 / 运筹优化	22

第 3 / 14 章 • 爬虫

库	一句话中文解释	章节
requests	发 HTTP 请求（抓网页、调接口）	3
beautifulsoup4 + lxml	从 HTML 里挑数据	3
sqlite3	单文件数据库（存爬到的数据）	3 / 20
playwright	开真浏览器，抓 JS 页面	3
scrapy	爬虫框架（大规模抓取）	3
tenacity	自动重试	3 / 8
tqdm	进度条	3 / 8 / 21
httpx	异步 HTTP（高并发）	14
selenium	老牌浏览器自动化	14
parsel	XPath / CSS 提取	14
pyquery	jQuery 风格解析	14
fake-useragent	随机 User-Agent	14
robobrowser	简化表单提交	14

第 4 / 15 / 16 章 • 网络与安全 [!]（仅自有设备 / 授权 / 靶场）

库	一句话中文解释	章节
socket	网络编程的地基（TCP/UDP）	4
hashlib / hmac	哈希指纹 / 带密钥签名	4
secrets	安全随机（生成 Token / 验证码）	4
bcrypt / argon2-cffi	密码哈希（**存密码必须用它**）	4
cryptography	现代加解密（AES / RSA）	4
bandit	扫自己代码的漏洞	4 / 16

库	一句话中文解释	章节
pip-audit	扫依赖库的 CVE 漏洞	4 / 16
yara-python	恶意样本特征匹配（**防守核心**）	4 / 16
faker	生成假测试数据	4 / 15
python-nmap	端口探测（只扫自己）	15
scapy	构造 / 嗅探网络包	15
dnspython	DNS 查询	15
python-whois	域名注册信息	15
pyshark	解析抓包文件	15
psutil	系统与网络状态监控	15
mitmproxy	中间人代理（调试自己的 App）	15
pyelftools / pefile	解析 ELF / PE 可执行文件	16
capstone	反汇编（机器码 → 汇编）	16
unicorn	CPU 模拟执行	16
angr	符号执行	16
pwntools	CTF 工具箱（**只支持 Linux/macOS**）	16
ssdeep / py-tlsh	模糊哈希（找相似样本）	16
geoip2	IP → 地理位置	16
elasticsearch	日志检索（SIEM）	16

第 5 / 17 章 • 中文处理与 AI

库	一句话中文解释	章节
jieba	中文分词	5
wordcloud	词云图	5 / 17
snownlp	中文情感分析	5 / 17
pypinyin	汉字转拼音	17
scikit-learn	传统机器学习（分类/回归/聚类）	5
xgboost / lightgbm / catboost	梯度提升树（表格数据最强）	17
transformers	几行加载 BERT / GPT	17
sentence-transformers	句向量、语义相似度	17
datasets	一行加载公开数据集	17
tiktoken	数 token（算 API 费用）	17
accelerate	多卡 / 混合精度训练	17
peft	LoRA 等高效微调	17
langchain / llama-index	RAG 与 Agent 编排	17
hanlp / ltp	专业中文 NLP	17
openai	调用大模型 API	5

第 6 / 18 章 • Web 后端

库	一句话中文解释	章节
FastAPI	高性能接口 + **自动生成文档**	6
Flask	最轻量的 Web 框架	18
Django	全家桶（自带后台、用户系统）	18
uvicorn / gunicorn	ASGI / WSGI 服务器	18
pydantic	数据校验（请求参数 / 配置）	6 / 18
SQLAlchemy	ORM：用类操作数据库	18 / 20
jinja2	模板引擎（渲染 HTML）	18
starlette	轻量 ASGI 框架	18
streamlit	几行代码做数据网页	18
gradio	几行代码做 AI 演示界面	18

第 7 / 12 / 19 章 • 办公、图像、文档

库	一句话中文解释	章节
openpyxl	读写 Excel(.xlsx)	7
xlrd	读老式 .xls	13
python-docx	读写 Word	7
python-pptx	读写 PPT	19
pypdf	PDF 拆分 / 合并 / 加密	7 / 19
pdfplumber	提取 PDF 文字与表格	7 / 19
PyMuPDF（pymupdf）	PDF 转图片、极速提取	19
reportlab	生成 PDF	19
weasyprint	HTML → PDF	19
Pillow	图像基础处理（裁剪/水印/转格式）	12
OpenCV（opencv-python）	计算机视觉（人脸/边缘/视频）	12
pytesseract / easyocr / paddleocr / cnocr	OCR 文字识别	12
moviepy	视频剪辑	12
ffmpeg-python	组装 ffmpeg 命令	12
pydub	音频切片与转换	12
mutagen	音频元数据（ID3）	12
librosa	音频特征提取	12
whisper	语音转文字	12
qrcode / pyzbar	生成 / 识别二维码条码	12
pyautogui	桌面自动化（鼠标键盘）	19
pywinauto	Windows 控件级自动化	19
schedule / APScheduler / python-crontab	定时任务	19
pyinstaller	打包成 exe	19

第 20 章 • 数据库

库	一句话中文解释	章节
sqlite3	单文件数据库（首选）	3 / 20
pymysql	连 MySQL	20
psycopg	连 PostgreSQL	20
pymongo	连 MongoDB（文档型）	20
redis	内存缓存 / 队列 / 计数	20
peewee / tortoise-orm	轻量 ORM	20
alembic	数据库迁移（改表不丢数据）	20
duckdb	直接用 SQL 查 CSV / Parquet	20

第 8 / 21 章 • 工程化

库	一句话中文解释	章节
pytest	自动化测试	8
pytest-cov	测试覆盖率	21
ruff	检查 + 格式化（一个顶三个）	8 / 21
black / isort / flake8	格式化 / import 排序 / 风格检查	21
mypy	静态类型检查	21
pre-commit	提交前自动检查	21
loguru	一行搞定的漂亮日志	8 / 21
icecream	调试打印（自动带变量名）	21
tqdm	进度条	8 / 21
rich	终端彩色输出、表格	21
typer / click	命令行工具框架	21
python-dotenv	从 .env 读密钥	21
PyYAML（pyyaml）	读写 YAML 配置	21
orjson	更快的 JSON	21
ipython	增强版交互解释器	21

> **一共 120+ 个库，覆盖 10 大方向。** 你不需要全学——按第 9 章的路线挑一条开始就行。

写在最后

回到最开始那个问题：**"学完语法之后该干什么？"**

答案就是这篇文章里的路线：

1. **先挑一个"马上能看到效果"的库**（比如 openpyxl 生成一张表）
2. **照着例子敲一遍，看到一样的输出**
3. **改一改参数，看哪里变了**
4. **把它用到你自己的重复劳动上**

编程能力不是"学"出来的，是"用"出来的。

> ● 配套资料：
> - 《常用第三方库速查.md》—— 120+ 个库的分类速查（想用某个库时翻它）
> - 《爬虫实战教程.md》—— 爬虫方向的完整实战（含可运行项目）
> - 《知识点文案.md》—— Python 基础知识点与错题本

---

*（本教程由 AI
助手整理，所有"运行结果"均来自实际运行；网络安全部分请只在自有设备、授权目标或靶场中练习。）*